Microcredencial Git

CI/CD con Github Actions

Índice

1. Introducción teórica
2. GitHub Actions

Dev vs Ops

👨‍💻 Desarrollo (Dev)

- **Objetivo:** Crear nuevas funcionalidades
- **Mentalidad:** "Cambio y velocidad"
- **Métricas:** Velocidad de desarrollo, nuevas features
- **Responsabilidad:** Hasta el deployment

"Funciona en mi máquina" 🤷‍♂️

🔧 Operaciones (Ops)

- **Objetivo:** Mantener sistemas estables y seguros
- **Mentalidad:** "Estabilidad y control"
- **Métricas:** Uptime, rendimiento, seguridad
- **Responsabilidad:** Producción y mantenimiento

"No toques nada que funcione" 🚫

⚔️

Conflicto natural entre velocidad y estabilidad

DevOps

DevOps

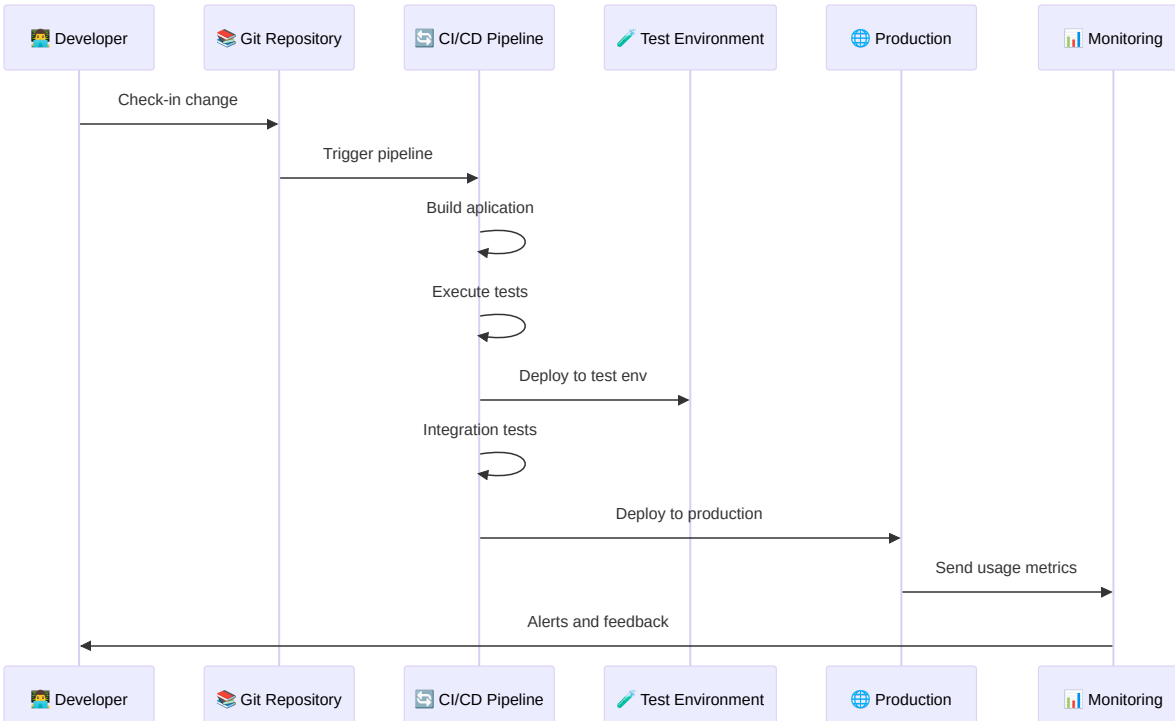
DevOps es un conjunto de prácticas, técnicas y herramientas que unifica los equipos de desarrollo de software (Dev) y operaciones (Ops).

- **Objetivo principal:**
  - Reducir tiempos de entrega de software de calidad
- **Rompe las barreras entre:**
  - Desarrollo (Dev) 👨‍💻
  - Operaciones (Ops) 🔧

Dev + Ops = DevOps

"You build it, you run it"

DevOps en la Práctica: Ejemplo



Integración Continua (CI)

+

Despliegue Continuo (CD)

🤖

Automatización total

Del commit a producción

✨ 🚀 ⚡

Herramientas del Ecosistema DevOps

🔧 Desarrollo

- Git, GitHub, GitLab
- IDE/Editors
- Jira, Trello

🏗️ Build & CI/CD

- Jenkins, GitHub Actions
- GitLab CI, Azure DevOps
- Travis CI, CircleCI

🧪 Testing

- JUnit, Jest, Selenium
- SonarQube
- Postman, Newman

📦 Containerización

- Docker
- Kubernetes
- Helm

☁️ Cloud & Infraestructura

- AWS, Azure, GCP
- Terraform, Ansible
- Vagrant

📊 Monitorización

- Prometheus, Grafana
- ELK Stack
- New Relic, Datadog

🎯 El objetivo no es usar todas las herramientas, sino elegir las que mejor se adapten a tu contexto

Integración / Despliegue Continuo

Integración Continua (CI)

Práctica de integrar cambios de código de forma frecuente en un repositorio compartido, ejecutando automáticamente build y pruebas para detectar errores lo antes posible.

Despliegue Continuo (CD)

Práctica de publicar automáticamente en producción cada cambio que supera las validaciones del pipeline, sin intervención manual en la fase de despliegue.

Github Actions

GitHub Actions es la plataforma de automatización de GitHub para definir workflows (CI/CD) en YAML que se ejecutan en respuesta a eventos del repositorio.

🏃 Runners

Máquinas (hosted o self-hosted) donde se ejecutan los jobs y sus steps. Máquinas disponibles.

⚙️ Workflows

Un proceso que ejecuta jobs cuando ocurre un evento.

📣 Events

Actividad que lanza un workflow (push, pull_request, schedule, etc.).

🧩 Jobs

Conjunto de steps que se ejecutan en una máquina (runner) distinto. Pueden ejecutarse en paralelo o secuencialmente.

🪜 Steps

Pasos individuales dentro de un job. Pueden ser un shell script o un Action. Se ejecutan de manera secuencial.

🔨 Actions

Acciones reutilizables que encapsulan tareas (publicadas en Marketplace o locales).

> "Cada workflow se define con un archivo `*.yml` en el directorio `.github/workflows/` del repositorio, que especifica eventos, jobs, steps, etc."

Workflow básico

```yaml
name: Java Maven Build

on:
  push:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Clonar repositorio
        uses: actions/checkout@v4

      - name: Configurar JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: 'maven'

      - name: Compilar y empaquetar
        run: mvn clean package
```

Nombre del workflow, se verá en la pestaña Actions en GitHub.

Evento: El workflow se dispara solo al hacer push en la rama main.

Runner: Se define el job `build`, que se ejecutará en una máquina virtual Linux.

Acción `Checkout`: ejecuta `git clone`.

Instalación del JDK 21 con distribución Temurin y caché de dependencias.

Ejecución directa de Maven para compilar y empaquetar el proyecto.

Step

```yaml
- name: Empaquetar proyecto con Maven
  id: maven_package
  if: success() # por defecto siempre es success
  run: mvn clean package -DskipTests
  working-directory: ./
  shell: bash
  env:
    MAVEN_OPTS: "-Xmx1024m"
  continue-on-error: true
  timeout-minutes: 15
```

Cada `Step` se ejecuta con un proceso shell nuevo, desde el directorio raiz. Usar comando `cd` o establecer `working-directory`.

Pueden ejecutarse condicionalmente con `if`:

- `success()`: se ejecuta si el Step anterior fue exitoso
- `failure()`: se ejecuta si algún Step anterior falló
- `always()`: se ejecuta siempre
- `cancelled()`: se ejecuta si el workflow se canceló
- `steps.maven_package.outcome == 'failure'`

: comprobando el estado de un step concreto

Definición de variables de entorno `env`:

- A nivel del Step
- A nivel del job
- A nivel del Workflow

Ejercicio 1.1

Creación de un `Workflow` sencillo

1. Descargar el código fuente de una aplicación java sencilla en este enlace
2. Crear un repositorio en GitHub para alojar el código de esa aplicación.
3. Activar GitHub Actions en el repositorio: `Settings → Actions → General → Allow all actions`
4. Crear un `Workflow` de GitHub Actions para compilar, ejecutar las pruebas y empaquetar el programa en un archivo `jar`.
   - El Workflow tiene que ejecutarse al hacer un push en la rama `main`
   - La aplicación utiliza el gestor Maven.
     - Compilación: `mvn clean compile`
     - Ejecución de las pruebas: `mvn clean test`
     - Empaquetamiento: `mvn clean package`
5. Comprobar que el `Workflow` se ejecuta cuando se hace un push en la rama `main`

Ejercicio 1.2

Comprobar cómo se visualiza un `Workflow` que falla.

1. Crear una rama nueva, y comprobar que cuando se hace un `push` de esa rama, el `Workflow` no se ejecuta.
2. Modificar el `Workflow` para también se ejecute en la rama nueva.
3. Modificar algun fichero java para añadir algún tipo de error de compilación. Comprobar que el `Workflow` falla.
4. Rehacer el cambio anterior. Comprobar que ahora el `Workflow` funciona.
5. Modificar cualquier prueba para que fallen. Se encuentran en el directorio `src/test/java`. Comprobar que el `Workflow` vuelve a fallar.

Ejercicio 1.3

Recuperar artefactos que se generan en GitHub Actions.

1. La ejecución de Maven genera una serie de artefactos que muestran información detallada del proceso de compilación, pruebas o empaquetado. Estos artefactos se generan en el directorio `target`
   - Comprobar que el directorio `target` esta siendo ignorado por git (fichero `.gitignore`)
2. Recuperar estos artefactos utilizando la acción `actions/upload-artifact@v4`. Tiene dos parámetros:
   - path: directorio que queremos recuperar
   - name: nombre que le vamos a dar a este artefacto (puede ser cualquiera)
3. Hacer que el `Step` que sube los artefactos sólo ocurra cuando las pruebas fallen
   - Para hacer esto, lo recomendable es tener un `Step` que únicamente ejecute las pruebas.
   - Subir sólo los artefactos localizados en el directorio `target/surefire-reports/`

Dependencias entre Workflows

Podemos lanzar un `Workflow` cuando termina otro utilizando `workflow_run`

```yaml
# Workflow receptor (por ejemplo: downstream.yml)
on:
  workflow_run:
    workflows: ["CI Build"]
    types: [completed]

jobs:
  run-after-build:
    if: github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - name: Notificar o continuar pipeline
        run: echo "El workflow 'CI Build' finalizó correctamente."
```

- Este `Workflow` se lanzará cuando el `Workflow` `"CI Build"` termine, aunque este falle.
- Podemos poner una condicón `if` al `job` para que se ejecute sólo cuando el `Workflow` haya finalizado exitosamente.

Dependencias entre Jobs

Por defecto los `jobs` dentro de un `workflow` se ejecutan en paralelo. Con `needs` podemos establecer dependencias entre `jobs`.

```yaml
jobs:
  compile:
    runs-on: ubuntu-latest
    steps:
      - name: Compilar
        run: |
          mvn clean compile

  test:
    needs: compile
    runs-on: ubuntu-latest
    steps:
      - name: Ejecutar pruebas si la compilación funcionó
        run: |
          mvn clean test
```

- El `job` `test` se ejecuta cuando `compile` termina exitosamente.
- Podemos hacer que un `job` se ejecute cuando otro falle, añadiendo una clausula `if` al `job`
  - `- if failure()`

Matrices (`strategy.matrix`)

`strategy.matrix` permite ejecutar el mismo job varias veces, con distintas configuraciones. Típicamente para probar distintos sistemas operativos o versiones de librerías.

```yaml
jobs:
  build-and-test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        java: [11, 17]
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: ${{ matrix.java }}

      - name: Build and test
        run: ...
```

- Este ejemplo ejecuta el `job` `build-and-test` 6 veces.
  - 2 versiones de Java (11, 17)
  - 3 sistemas operativos distintos (Linux, Windows, macOS)
- Por defecto, si un `job` falla, se cancela toda la matriz.
  - Para no cancelar la matriz, usar `fail-fast: false`.

Ejercicio 1.4 — Dependencias entre jobs

Crear un workflow que contenga dos jobs: `compile` y `test`.

Instrucciones:

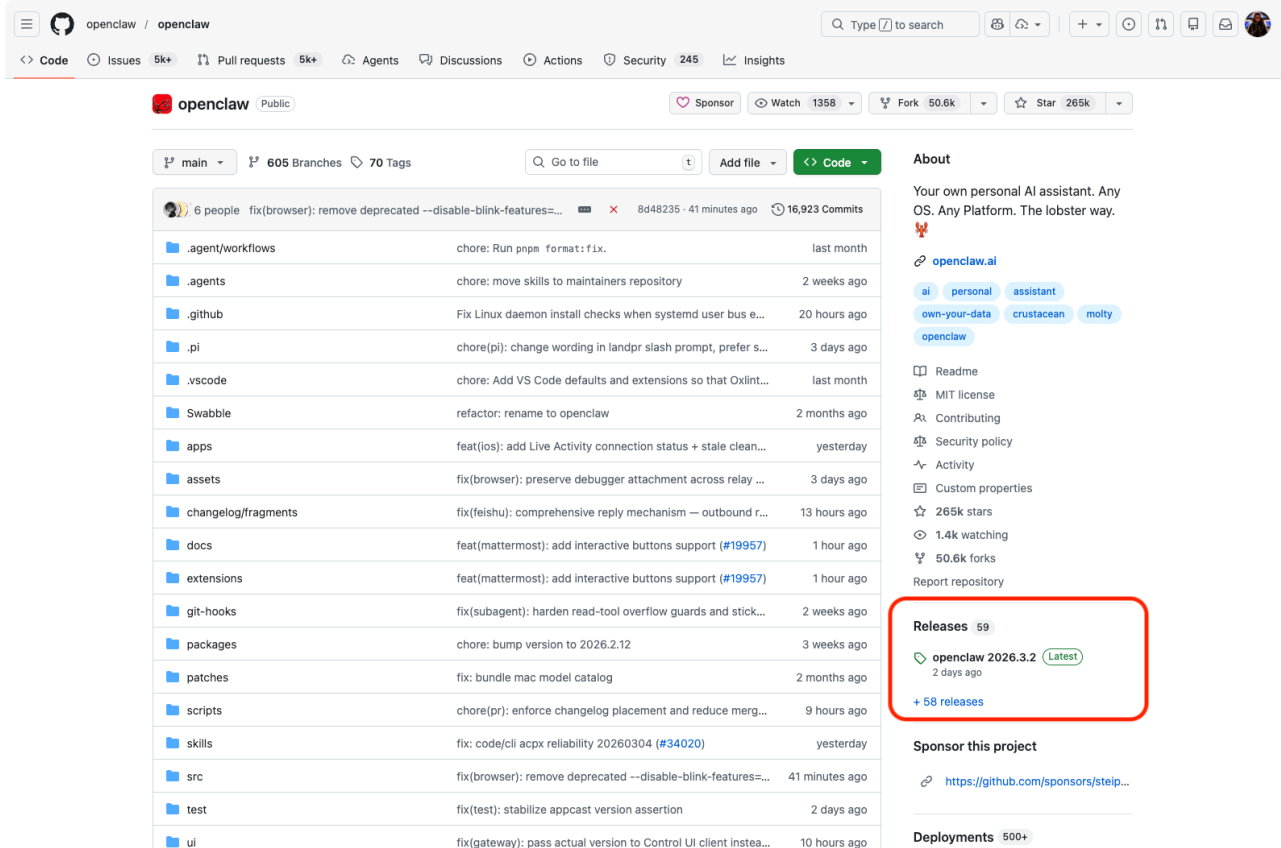
1. En el Ejercicio 1.3, todo se ejecutaba en el mismo `job`. Ahora tenemos que modificar ese `workflow` para que haga el trabajo en varios `jobs`.
   - El `job` `compile` debe compilar el proyecto:
   - El `job` `test` debe ejecutar las pruebas. Sólo debe ejecutarse si el `job` `compile` fue exitoso. La recuperación de artefactos puede hacerse en este `job`.
2. Verificar en la pestaña Actions que `test` se ejecuta solo si `compile` finaliza correctamente.

Ejercicio 1.5 — Matrix

Crear un workflow para compilar y ejecutar pruebas en macOS, Windows y Linux usando `strategy.matrix`.

Instrucciones:

1. Modificar el `workflow` del ejercicio 1.4 para:
   - Compilar con java versión 17 y linux.
   - Ejecutar las pruebas en macOS, windows y linux, utilizando JDK versión 17, 21 y 25.
2. Comprobar cómo se muestran los `jobs` en la pestaña `Actions` del repositorio.

GitHub Releases

Publicar una versión distribuible de la aplicación dentro de GitHub